

DESIGN DOCUMENT

appsec-review v3

A repeatable, hands-off application security review process

VERSION	STATUS	DATE	OWNER
0.2	Draft	2026-09-27	W. Sollers
AUDIENCE	Security leadership, engineering directors, vendor management		

Template. Amber *TBD* marks are fill-ins; compile with `\usepackage[final]{appsec-house}` to hide this line.

BOTTOM LINE

Every security review should run the same way, whoever starts it and whatever the target. v3 turns the review into a pipeline: one command takes a repository from checkout to a scored report, with no one running tools by hand along the way. Each step uses pinned tools and fixed rules, every finding has to survive an attempt to disprove it, and anything the pipeline could not examine is listed in the report instead of being passed over. People review the finished report; they do not operate the review.

COMMAND TO RUN

1

Checkout to report, unattended

SEEDED DEFECTS FOUND

4/4

Notepad++ 8.5.6; stock Cod-eQL found 0/4

FALSE FINDINGS SHIPPED

0

EASTL baseline: 8 custom-pack hits, none real

PINNED TOOL IMAGES

13

Same versions every run, no network at scan time

What this means for the organization

- **Repeatable.** The same target at the same commit produces the same evidence and the same report. Results hold up when a vendor disputes them.
- **Hands-off.** Build, scanning, analysis, refutation, verification and scoring run as one flow. Reviewer time goes to judging the report, not running tools.
- **Honest about coverage.** A step that did not run is reported as a gap. The report can only call a target clean when enough of the review actually happened.
- **Not tied to one person.** The process lives in the pipeline, so reviews do not stop when a particular reviewer is unavailable.

Contents

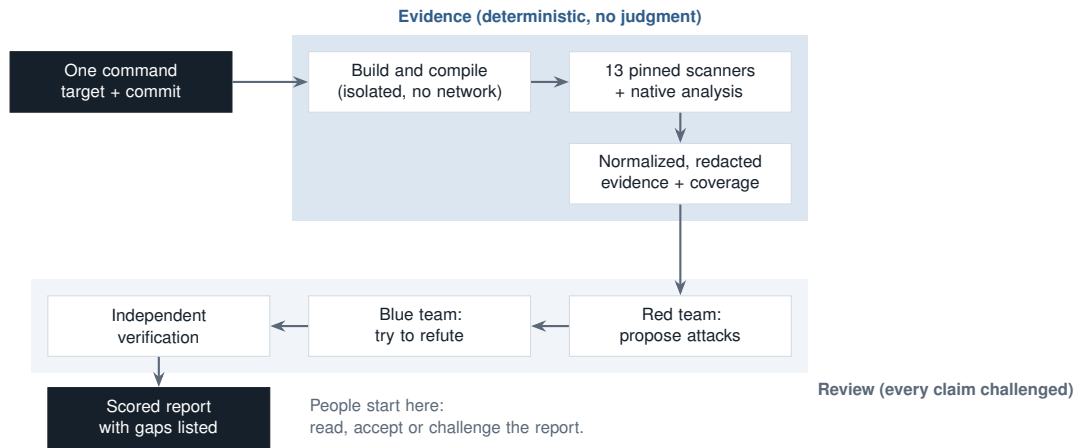
1	Why change	2
2	How the process runs	2
3	Options considered	3
4	Where we are	4
5	Risks	4
6	How we will measure success	4
A	Technical detail	5
A.1	Scoring	5
A.2	Native code analysis tiers	5
B	Revision history	5

1 Why change

TODAY	CONSEQUENCE
A reviewer runs tools by hand from a script collection	Each review depends on who ran it and how; results differ run to run.
Scanner output copied into reports	False positives reach vendors and erode trust; real issues hide in the noise.
Coverage not recorded	A report cannot say what was <i>not</i> checked.
Steps live in one person's habits	Reviews queue behind one reviewer and stop when they are away.

2 How the process runs

A review is a fixed sequence of jobs started by one command. Nothing in the sequence waits for a person. The evidence side gathers facts and never makes a claim; the review side never promotes a tool hit to a finding without trying to refute it and verifying what survives.



What makes it repeatable

1. **Pinned everything.** Tool versions, container images and reference data are fixed per run; nothing is looked up live while scanning.
2. **Fixed contracts.** Every job has declared inputs, outputs and a pass/fail check. A job that fails stops with a recorded reason and can be resumed from that point.
3. **Evidence-first.** Every finding cites a file and line or a tool output, so anyone can trace it back.
4. **Target content is data.** Nothing in the code under review can steer the process.

Where people stay in the loop

POINT	WHY A PERSON
Choosing the target	Scope and authority to review belong to the business.
Reading the report	Accepting risk is a business decision, not a pipeline output.
Vendor disputes	The evidence trail supports the conversation; a person owns it.

3 Options considered

OPTION	FOR	AGAINST
A. Keep the manual tool-box	Nothing new to build.	Not repeatable; depends on one reviewer; false positives reach vendors.
B. Adopt a commercial SAST platform	Supported; dashboards.	Breadth scanning only: stock tooling missed all four seeded memory bugs. No refutation step; still needs a person to triage.
C. v3 RECOMMENDED pipeline	Runs end to end without an operator; measured recall on native code; refutation built in; reports gaps.	We maintain it.

4 Where we are

MILESTONE	STATUS	EVIDENCE
Pinned tool images (13)	● Done	Built and smoke-tested on both hosts.
Orchestration runtime	● Done	Qualified; runs the job graph unattended.
Unattended run, intake to build resolution	● Done	Acceptance stages 1–13 qualified on the test target.
Scanners as pipeline jobs	● In progress	Secrets and config scans first; vulnerability matching waits on an offline database mirror.
Review lanes run by the pipeline	● In progress	Run manually on EASTL; not yet driven end to end.
Report generation	● Proposed	Template on branch <code>report-template</code> ; format decision open.
First fully hands-off review	● Not started	Target: TBD: engagement, by TBD: date.

5 Risks

RISK	L	I		MITIGATION
Automated reasoning is wrong or overconfident	2	3	H	Blue-team refutation and independent verification before any High or Critical finding; evidence must resolve.
Hostile code in the target attacks the build	2	3	H	Builds run with no network, read-only root, dropped capabilities, non-root user.
A step fails silently and the report looks clean	1	3	M	Every step reports a status; missing coverage lowers assurance and blocks a clean rating.
Vulnerability data goes stale offline	2	2	M	Mirrored snapshots with recorded dates; stale data is reported as a gap.
New languages or build systems	2	2	M	Per-language build images; unsupported builds fall back to source-only analysis with capped confidence.

L = likelihood, I = impact, each 1 (low) to 3 (high).

6 How we will measure success

Each measure is read from run records, so it is checked by the pipeline rather than self-reported.

MEASURE	TARGET	HOW MEASURED
Reviews completed with no manual step	100%	Run records show no operator action between start and report.
Same result on re-run	100%	Re-run a finished review at the same commit; compare evidence and findings.
Recall on seeded defects	$\geq$ TBD: 90%	Held-out seeded targets, not used to tune rules.
False findings in delivered reports	0	Vendor disputes upheld after delivery.
Reports with every gap disclosed	100%	Automated check against the run's coverage record.

A Technical detail

A.1 Scoring

Each finding's priority starts from its CVSS 4.0 base score and is scaled by how strong the evidence is, whether it survived verification, whether the code is reachable, and exploitation signals:

$$P_f = \min(10, S_f \cdot e_f \cdot v_f \cdot r_f \cdot t_f)$$

The overall rating is the band of the highest non-refuted priority. A Low or clean rating is only issued when process assurance, the share of the review that actually ran, is at least 0.85; otherwise the rating is *Indeterminate*.

A.2 Native code analysis tiers

A	Every translation unit compiles: full compiled analysis. Strongest claims allowed.
B	Some units compile: compiled analysis where possible, source analysis elsewhere; coverage recorded per unit.
C	Nothing compiles: source analysis only; memory-safety confidence capped.

B Revision history

VERSION	DATE	AUTHOR	CHANGE
0.2	2026-09-27	W. Sollers	Repositioned as a repeatable, hands-off process; removed cost and asks.
0.1	2026-09-27	W. Sollers	First draft from the template.